

Churn Analysis and Customer Intelligence

```
[ ]: # Churn Analysis and Customer Intelligence
```

```
[2]: # Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
```

1. Import database / data

```
[ ]: # NOTE:
# If you encounter any issues importing the customer_churn.db file,
# Run the code below to create the database locally from the provided Excel file OR, can directly import data & start working.
```

```
# Otherwise, skip this step and proceed to the next section: Data Import
```

```
# Excel file path
excel_file = 'customer_churn_data_raw.xlsx'

# Create SQLite database connection
conn = sqlite3.connect('customer_churn.db')

# Read all sheet names from Excel file
excel_data = pd.ExcelFile(excel_file)

# Loop through each sheet and store as separate table
for sheet in excel_data.sheet_names():
    df = pd.read_excel(excel_file, sheet_name=sheet) # Read sheet into dataframe
    # Write dataframe to SQLite table
    df.to_sql(
        name=sheet,          # Table name = sheet name
        con=conn,
        if_exists='replace', # Replace table if already exists
        index=False
    )

# Close connection
conn.close()
```

```
print("All sheets successfully converted into SQLite tables.")
```

```
[ ]: # Data Import: db file to pandas, storing each table to a separate df
```

```
# Connect to SQLite database
conn = sqlite3.connect('customer_churn.db')

# sql query to get all table names
sql_query = """
SELECT name
FROM sqlite_master
WHERE type='table';
"""

# read sql query in pandas
tables = pd.read_sql(sql_query, conn)

# create dataframe for each table
for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}", conn) # Read table into dataframe
    globals()[f"df_{table_name}"] = df # Create dynamic dataframe name
    print(f"Created dataframe: df_{table_name}")

# Close connection
conn.close()
```

```
Created dataframe: df_db_customer
```

```
Created dataframe: df_db_subscription
```

```
Created dataframe: df_db_support
```

```
[ ]: # Print table names and column names
conn = sqlite3.connect('customer_churn.db')
```

```
for table_name in tables['name']:
    print(f"Table Name: {table_name}")
    # Get column information
    columns_query = f"PRAGMA table_info({table_name});"
    columns = pd.read_sql(columns_query, conn)
    print(f"Columns: {columns['name'].tolist()}")

# Close connection
conn.close()
```

```
Table Name: db_customer
```

```
Columns:
['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']
```

```
Table Name: db_subscription
```

```
Columns:
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cstv', 'churn_score']
```

```
Table Name: db_support
```

```
Columns:
['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']
```

```
[ ]: # PRAGMA is a special command in SQLite used to:
# Inspect db information, control db settings, retrieve metadata about tables
```

```
[ ]:
```

2. Data cleaning

```
[ ]: # df_db_customer.head() # customer table
df_db_customer.tail()
```

```
[8]:
```

	customerid	name	country	state	gender	dob	interests	pincode
16	0020-IDNXP	rikm	India	Meghalaya	Female	1994-08-19 00:00:00	None	None
17	0021-HKGGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	None	None
18	0022-TCJGJ	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	None	None
19	0023-HGHWL	nishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	None	None
20	0023-UVUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	None	None

```
[ ]: df_db_customer.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype
---  ---
 0   customerid  21 non-null    object
 1   name        21 non-null    object
 2   country     18 non-null    object
 3   state       21 non-null    object
 4   gender      21 non-null    object
 5   dob         21 non-null    object
 6   interests   4 non-null     object
 7   pincode     0 non-null     object
dtypes: object(8)
memory usage: 1.4+ KB
```

```
[ ]: # a. rename col - name to customer_name
```

```
# b. drop columns - interest and pincode
```

```
# c. change data type - dob
```

```
# d. data standardization - gender
```

```
# e. fix missing values (using existing data) - country
```

a. Rename Col

```
[ ]: # a. rename col - name to customer_name
```

```
df_db_customer.rename(columns = {'name': 'customer_name'}, inplace=True)
```

b. Drop columns

```
[ ]: # b. drop columns - interest and pincode
```

```
# df_db_customer.drop(df_db_customer.columns[-2:], axis=1)
```

```
# df_db_customer.drop(df_db_customer.columns[6:], axis=1)
```

```
df_db_customer.drop(columns=['interests', 'pincode'], inplace=True) # using col name
```

```
[ ]: df_db_customer.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  ---
 0   customerid  21 non-null    object
 1   customer_name  21 non-null    object
 2   country     18 non-null    object
 3   state       21 non-null    object
 4   gender      21 non-null    object
 5   dob         21 non-null    object
dtypes: object(6)
memory usage: 1.3+ KB
```

```
[ ]: # c. change data type - dob

df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])
```

```
[ ]: # d. data standardization - gender

# df_db_customer['gender'].unique()
df_db_customer['gender'] = df_db_customer['gender'].replace({'Men' : 'Male', 'Women' : 'Female'})
```

```
[16]: array(['Male', 'Female'], dtype=object)
```

```
[ ]: # e. fix missing values - country

# df_db_customer['country'].isna().sum()
df_db_customer[df_db_customer['country'].isna()]
```

[27]:	customerid	customer_name	country	state	gender	dob
5	0013-MHZWF	durga	None	Delhi	Female	1988-12-10
8	0015-UOOCJ	maya	None	Kathmandu	Female	1985-07-07
12	0018-NYROU	chitra	None	Telangana	Female	2004-12-01

```
[ ]: # country and state - unique value pair

# Creating state + country map from non-null rows
state_country_mapping = df_db_customer.dropna(subset=['country']).set_index('state')['country'].to_dict()

# Fill the missing country using state
df_db_customer['country'] = df_db_customer['country'].fillna(df_db_customer['state'].map(state_country_mapping))

[ ]: df_db_customer[df_db_customer['country'].isna()] # no null value in country col
```

```
[19]: customerid customer_name country state gender dob
```

```
[ ]: df_db_subscription.head() # subscription table
```

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract_type	cancellation_date	cancellation_reason	monthly_charges	chtr_churn_score
0	0002-ORFBO	2021-03-15	Referral	2025-03-15	Standard	Annual	None	None	13.99	627
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	None	None	6.99	2110
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	None	None	22.99	1725
4	0013-EXCHZ	2023-01-05	Referral	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195

```
df_subscriber.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   customerid             21 non-null    object
 1   subscription_start_date 21 non-null    object
 2   subscription_type       21 non-null    object
 3   renewal_date           21 non-null    object
 4   plan_type              21 non-null    object
 5   contract_type          21 non-null    object
 6   cancellation_date       6 non-null     object
 7   cancellation_reason     6 non-null     object
 8   monthly_charges        21 non-null    float64
 9   city                   21 non-null    int64
10   churn_score             21 non-null    int64
dtypes: float64(1), int64(2), object(8)
memory usage: 1.9+ KB
```

```

# change date type to date - subscription_start_date, renewal_date, cancellation_date
date_col = ['subscription_start_date', 'renewal_date', 'cancellation_date']

df_db_subscription[date_col] = df_db_subscription[date_col].apply(pd.to_datetime)
df_db_subscription.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  --
0   customerid             21 non-null     object
1   subscription_start_date 21 non-null     datetime64[ns]
2   subscription_type       21 non-null     object
3   renewal_date           21 non-null     datetime64[ns]
4   plan_type              21 non-null     object
5   contract_type          21 non-null     object
6   cancellation_date       4 non-null      datetime64[ns]
7   cancellation_reason     6 non-null      object
8   monthly_charges        21 non-null     float64
9   cdiv                   21 non-null     int64
10  churn_score             21 non-null     int64
dtypes: datetime64[ns](3), float64(1), int64(2), object(5)
memory usage: 1.8+ KB

```

```
[ ]: df_db_support.head() # support table
```

[2]:	customerid	complaint_date	escalations	csat_score	col_1	comment
0	0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
1	0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demandd refund
2	0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	None
3	0013-MHZWF	2025-01-18 00:00:00	N	90	None	guidance to renew
4	0013-SMEOE	2024-11-01 00:00:00	N	30	None	None

```
> df_0.support.info()
<class 'pandas.core.frame.DataFrame'>
Int64Index: 9 entries, 0 to 8
Data columns (total 6 columns):
 #   Column                Non-Null Count  Dtype  
---  --
 0   customerid            9 non-null     object 
 1   complaint_date        9 non-null     object 
 2   escalations           9 non-null     object 
 3   cost_score            9 non-null     int64   
 4   col_3                 9 non-null     object 
 5   comment               4 non-null     object 
dtypes: int64(1), object(5)
memory usage: 564.0+ bytes
```

```
[ ]: # drop columns
df_db_support.drop(columns=['col_1', 'comment'], inplace=True)
```

```

# change data type to date - complaint_date

df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date'])
df_db_support.info()

<class 'pandas.core.frame.DataFrame'>
Int64Index: 9 entries, 0 to 8
Data columns (total 4 columns):
 #   Column                Non-Null Count  Dtype  ---
0   customerid            9 non-null      object
1   complaint_date        9 non-null      object
2   escalation            9 non-null      object
3   csat_score            9 non-null      int64
dtypes: datetime64[ns](1), int64(1), object(2)
memory usage: 416.0 bytes

```

Create a new col

```
[ ]: # create a new col using existing col - churn flag

# Customer is churned if cancellation_date is not null
df_db_subscription['churn_flag'] = np.where(df_db_subscription['cancellation_date'].notna(), 1, 0)
df_db_subscription.head()
```

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract_type	cancellation_date	cancellation_reason	monthly_charges	cltv	churn_score
0	0002-ORFSO	2020-03-15	Referral	2025-03-15	Standard	Annual	NaT	None	13.99	627	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	None	None	6.99	210	
3	0011-IGKFF	2019-05-10	Paid	2023-05-10	Premium	Annual	NaT	None	22.99	1725	
4	0013-EXCHZ	2023-01-05	Referral	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	

Merge Dataframes - JOINS

```
[ ]: # first file support table duplicates then merge
# Note: while merging df's always check the shape before and after
df = (df_db_subscription
      .merge(df_db_customer, on = 'customerId', how = 'left')
      .merge(df_db_support, on = 'customerId', how = 'left'))

[ ]: df_db_subscription.shape
[30]: (21, 12)
[ ]: df.shape
[31]: (23, 20)
[ ]: print(df_db_subscription.unique values', df_db_subscription['customerId'].nunique())
print(df_db_customer.unique values', df_db_customer['customerId'].nunique())
print(df_db_support.unique values', df_db_support['customerId'].nunique())
print(df_db_support all values', df_db_support['customerId'].size)
df_db_subscription unique value: 21
df_db_customer unique value: 11
df_db_support unique value: 7
df_db_support all value: 9

[ ]: df_db_support
[33]:
  customerId  complaint date  escalations  csat_score
0  0003-MNOFE      2024-08-28          N          60
1  0003-MNOFE      2024-08-28          Y          10
2  0013-EXCHZ      2024-01-20          Y          20
3  0013-MHZWF      2025-03-18          N          90
4  0013-SMEDE      2024-11-01          N          30
5  0017-IUDMW      2024-04-10          Y          25
6  0019-EFAEP      2024-09-27          Y          30
7  0022-TCICI      2024-09-13          Y          10
8  0022-TCICI      2024-09-14          N          90

[ ]: df_db_support['complaint_count'] = df_db_support.groupby('customerId')['customerId'].transform('count')

[ ]: df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('customerId', keep= 'last')

[ ]: df_db_support['customerId'].size
[36]: 7

[ ]: # merge df
df = (df_db_subscription
      .merge(df_db_customer, on = 'customerId', how = 'left')
      .merge(df_db_support, on = 'customerId', how = 'left'))

[ ]: df.shape
[38]: (21, 21)
```

Export dataframe

```
[ ]: # export dataframe to csv file
df.to_csv('exported_churn_data.csv', index=False)

[ ]:
```

Data Analysis

```
[ ]: # 1. Churn Rate
churn_rate = df['churn_flag'].mean()*100
print("Churn Rate = ", round(churn_rate,2), "%")
Churn Rate = 28.57 %

[ ]: # 2. Retention Rate
retention_rate = 100 - churn_rate
print("Retention Rate = ", round(retention_rate,2), "%")
Retention Rate = 71.43 %

[ ]: # 3. Churn by Plan type
churn_by_plan = (df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).reset_index(name='churn_rate_pct'))
print(churn_by_plan)
plan_type  churn_rate_pct
0  Basic          60.00
1  Premium       14.29
2  Standard       22.22

[ ]: # 4 a. Churn by state + sum(revenue) & count of users -- home work **
# 4 b. Churn by subscription type + sum(revenue) & count of users -- home work **

[ ]: # 5. ARPU - Avg Revenue per user
arpu = df['monthly_charges'].mean()
print("ARPU = ", round(arpu,2))
ARPU = 18.85

[ ]: # homework - calculate customer age

[ ]: # 6. Avg Customer Tenure
# count of days users has used our service : cancellation date else current date
today = pd.Timestamp.today()

df['tenure_days'] = np.where(
    df['cancellation_date'].notna(),

    (df['cancellation_date'] - df['subscription_start_date']).dt.days,

    (today - df['subscription_start_date']).dt.days
)

avg_tenure = df['tenure_days'].mean()
print("Avg Tenure (Days) = ", round(avg_tenure,0))
Avg Tenure (Days) = 1452 0

[ ]: # 7. Revenue at risk - revenue lost from churned users
revenue_at_risk = df[df['churn_flag']==1]['monthly_charges'].sum()
print("Revenue at Risk (Rs 'k') = ", revenue_at_risk)
Revenue at Risk (Rs 'k') = 73.94

[ ]: # 8. Escalation Rate
escalation_rate = (df['escalations']==Y').mean()*100
print("Escalation Rate = ", round(escalation_rate, 2), "%")
Escalation Rate = 19.05 %

[ ]: # 9. Avg Complaint Per User
avg_complaints = df['complaint_count'].sum() / df['customerId'].nunique()
print("Avg Complaints Per User = ", round(avg_complaints, 2))
Avg Complaints Per User = 0.43

[ ]: # 10. Correlation Escalation vs Churn
df['escalations'] = np.where(df['escalations'] == 'Y', 1, 0) # encoding string to int type
corr_df = df[['escalations', 'churn_flag']].dropna()

correlation = corr_df['escalations'].corr(df['churn_flag'])
print("Correlation between escalation vs churn is = ", round(correlation,2))
Correlation between escalation vs churn is = 0.77

[ ]: # 11. Create a column using existing col - Churn Risk
conditions = [
    (df['churn_score'] < 50),
    (df['churn_score'] >= 50 & (df['churn_score'] < 70),
    (df['churn_score'] >= 70)
]

choices = ['low', 'med', 'high']

df['churn_risk'] = np.select(conditions, choices, default='unknown')

[ ]:
```

4. Visualization using Matplotlib

```
[ ]: # best practice to create a copy then work on it
df_visual = df.copy()

[ ]: df_visual.columns
[36]: Index(['customerId', 'subscription_start_date', 'subscription_type',
'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
'cancellation_reason', 'monthly_charges', 'cls', 'churn_score',
'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
'complaint_date', 'escalations', 'csat_score', 'complaint_count',
'tenure_days', 'churn_risk'],
      dtype='object')

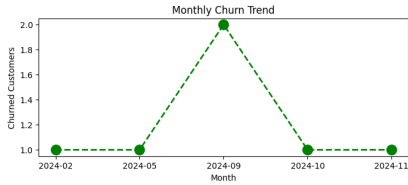
[ ]: # 4.1 Monthly Churn Trend (Time Series KPI)

df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M')

churn_trend = df_visual[df_visual['churn_flag'] == 1].groupby('cancellation_month').size()

plt.figure(figsize=(9,3))
plt.plot(churn_trend.index.astype(str), churn_trend.values, color='green', marker='o', linestyle='dashed', linewidth=2, markersize=12)
```

```
plt.title('Monthly Churn Trend')
plt.xlabel('Month')
plt.ylabel('Churned Customers')
plt.show()
```

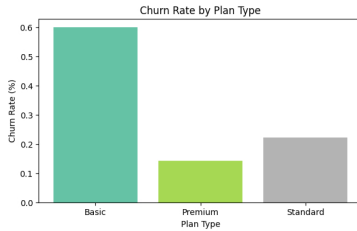


```
[ ]: # 4.2 Churn Rate by Plan type
churn_plan = df_visual.groupby('plan_type')['churn_flag'].mean()

# colors = ['yellow', 'purple', 'blue']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize=(7,4))
plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn Rate by Plan Type')
plt.xlabel('Plan Type')
plt.ylabel('Churn Rate (%)')
plt.show()
```

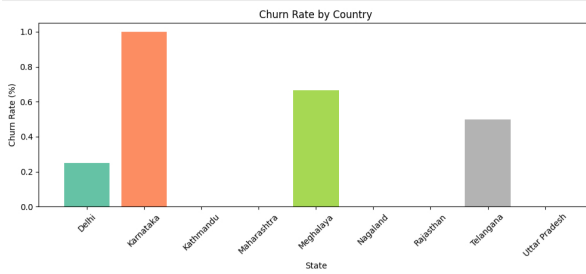


```
[ ]: # 4.3 Churn by States
churn_plan = df_visual.groupby('state')['churn_flag'].mean()

# colors = ['yellow', 'purple', 'blue']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize=(12,4))
plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn Rate by Country')
plt.xlabel('State')
plt.ylabel('Churn Rate (%)')
plt.xticks(rotation=45)
plt.show()
```



```
[ ]:
```

Visulaization using Seaborn

```
[ ]: # Heatmap (Correlation Matrix)
```

```
# encoding - convert str to numeric so that we can find corr between features***
```

```
[64]: Index(['customerid', 'subscription_start_date', 'subscription_type',
'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
'cancellation_reason', 'monthly_charges', 'churn', 'churn_score',
'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
'complaint_date', 'escalations', 'cat_score', 'complaint_count',
'tenure_days', 'churn_risk', 'cancellation_month'],
dtype='object')
```

```
[ ]: df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk', 'escalations']].head()
```

```
[64]: plan_type contract_type churn_score churn_flag churn_risk escalations
0 Standard Annual 12 0 low 0
1 Premium Annual 91 1 high 1
2 Basic Monthly 34 0 low 0
3 Premium Annual 8 0 low 0
4 Standard Monthly 88 1 high 1
```

```
[ ]: # remove warnings
import warnings
warnings.filterwarnings("ignore")
```

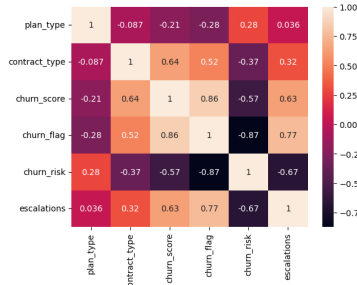
```
[ ]: # Incorrect method of encoding - as numbers are not assigned based on priority
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk', 'escalations']]

categorical_cols = ['plan_type', 'contract_type', 'churn_risk']

for col in categorical_cols:
    df_encoded[col] = df_encoded[col].astype('category').cat.codes
```

```
[ ]: # Heatmap (correlation matrix)
sns.heatmap(df_encoded.corr(), annot=True)
```

```
[64]: <axes: >
```

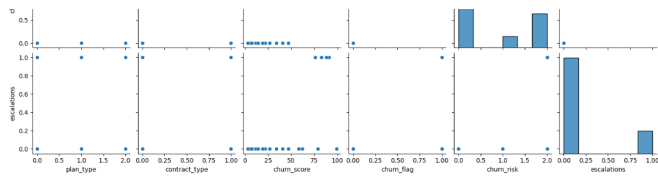


```
[ ]: print('plan_type :', df_visual['plan_type'].unique())
print('contract_type :', df_visual['contract_type'].unique())
print('churn_risk :', df_visual['churn_risk'].unique())

plan_type : ['Standard' 'Premium' 'Basic']
contract_type : ['Annual' 'Monthly']
churn_risk : ['low' 'high' 'med']
```

```
[ ]: # Correct method of encoding - based on priority
```

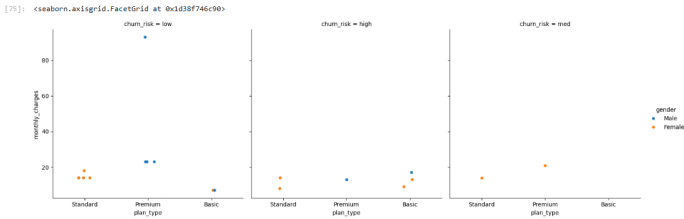




```
[ ]: df_visual.columns

[74]: Index(['customerid', 'subscription_start_date', 'subscription_type',
        'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
        'cancellation_reason', 'monthly_charges', 'city', 'churn_score',
        'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
        'complaint_date', 'escalations', 'cust_score', 'complaint_count',
        'tenure_days', 'churn_risk', 'cancellation_month'],
        dtype='object')

[ ]: # catplot/facgrid plot - multi-dim comparison
sns.catplot(data=df_visual,
            x='plan_type',
            y='monthly_charges',
            hue='gender',
            col='churn_risk')
```



```
[ ]:
```

Pivot table

```
[ ]: # Pivot Table
pd.pivot_table(
    df_visual,
    index='plan_type',
    values='churn_flag',
    aggfunc='mean'
)

[77]: churn_flag
plan_type
Basic      0.600000
Premium    0.142857
Standard   0.222222

[ ]: # pivot table using multiple cols and agg type
pd.pivot_table(
    df_visual,
    index='plan_type',
    values=['monthly_charges', 'customerid', 'churn_flag'],
    aggfunc=[
        'sum',
        'customerid',
        'churn_flag'
    ]
)
```

```
[78]: churn_flag customerid monthly_charges
plan_type
Basic      0.600000      5      52.95
Premium    0.142857      7     218.93
Standard   0.222222      9     123.91
```

```
[ ]:
```

Project Completed :)

```
# Project Completed Here. All the best! :)***
```

```
[ ]:
```

Working with SQL in Python

```
[4]: # create db in sql
conn = sqlite3.connect('test_database.sqlite')

# table details
conn.execute("CREATE TABLE users (first_name TEXT, country TEXT, budget INTEGER)")

# commit and save
conn.commit()

print("Created Table successfully!")
Created Table successfully!
```

```
[5]: # Insert data
cursor = conn.cursor()

# write sql query to insert records in sql table
cursor.execute(
    """
    INSERT INTO users VALUES
    ('Nishant', 'India', 5000),
    ('Ashish', 'France', 250000),
    ('Rahul', 'India', 35000)
    """
)

# commit and save
conn.commit()

print("Data Inserted successfully!")
Data Inserted successfully!
```

```
[6]: # check inserted data in table
conn = sqlite3.connect('test_database.sqlite')
query = """SELECT * FROM users"""

df_results = pd.read_sql(query, conn)
df_results.head()
```

```
[6]: first_name country budget
0 Nishant India 5000
1 Ashish France 250000
2 Rahul India 35000
```

```
[7]: # aggregation
query = """
SELECT country, SUM(budget) as total_budget
FROM users
GROUP BY country
"""

df_agg = pd.read_sql(query, conn)
df_agg
```

```
[7]: country total_budget
0 France 250000
1 India 40000
```

```
[ ]:
```

```
[ ]:
```

